

Date: 23/07/2025

Lab Practical #8:

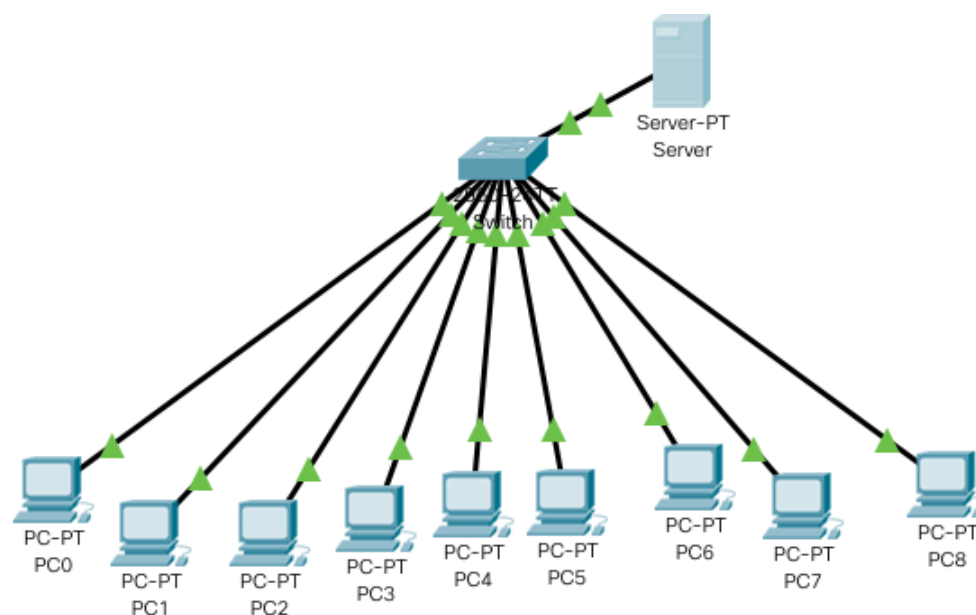
Study & Survey of Institute organization network infrastructure.

Practical Assignment #8:

1. Identify type of network in your institute. Draw a design of network in your institute (Any Lab/Floor/Building).

Lab No: C- 302

Topology: Star Topology



2. List how many network devices and types of cable used and give its details.

Network Devices:

1. **Router**
 - Connects different networks and routes data packets between them, such as between a local network and the internet.
2. **Switch**
 - Connects multiple devices within the same network and directs data to the correct destination device.
3. **Firewall**
 - Provides network security by monitoring and controlling incoming and outgoing traffic based on predetermined rules.
4. **Access Point (AP)**
 - Provides wireless network connectivity for devices to connect to a wired network.
5. **Network Interface Card (NIC)**
 - Hardware installed in devices to enable network connectivity.
6. **Modem**



Date: 23/07/2025

- Modulates and demodulates signals for internet access over telephone lines or other communication mediums.

7. Load Balancer

- Distributes network or application traffic across multiple servers to ensure reliability and performance.

Types of Cables:

1. Ethernet Cable (Twisted Pair - Cat5e, Cat6, Cat6a)

- Used for wired network connections; twisted pairs reduce interference and support high-speed data transfer.

2. Fiber Optic Cable

- Uses light to transmit data at very high speeds over long distances with minimal signal loss.

3. Coaxial Cable

- Used in some legacy network setups; carries data via electrical signals within a single conductor surrounded by shielding.

4. Console Cable (RJ45 to Serial)

- Used to connect network devices like switches and routers to a computer for configuration.

Date: 30/07/2025

Lab Practical #09:

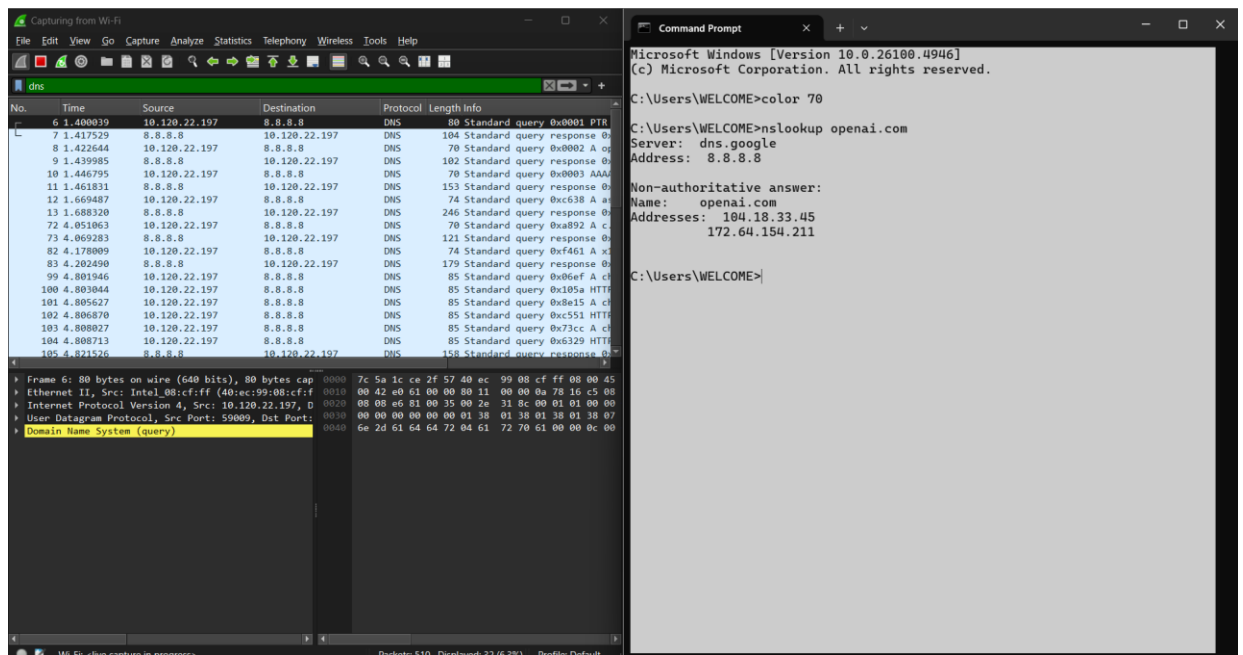
Study Packet capture and header analysis by Wireshark (HTTP, TCP, UDP, IP, etc.)

Practical Assignment #09:

1. Explain usage of Wireshark tool.

Wireshark is a network protocol analyzer used to capture and analyze network traffic in real time. It helps in troubleshooting network issues, monitoring performance, detecting security threats, and learning protocol behavior.

2. Packet capture and header analysis by Wireshark (HTTP, TCP, UDP, IP, etc.)



The screenshot displays two windows. On the left is the Wireshark network protocol analyzer, showing a packet capture from a Wi-Fi interface. The packet list pane shows a series of DNS queries and responses. The packet details pane for the selected packet (No. 6) shows the following structure:

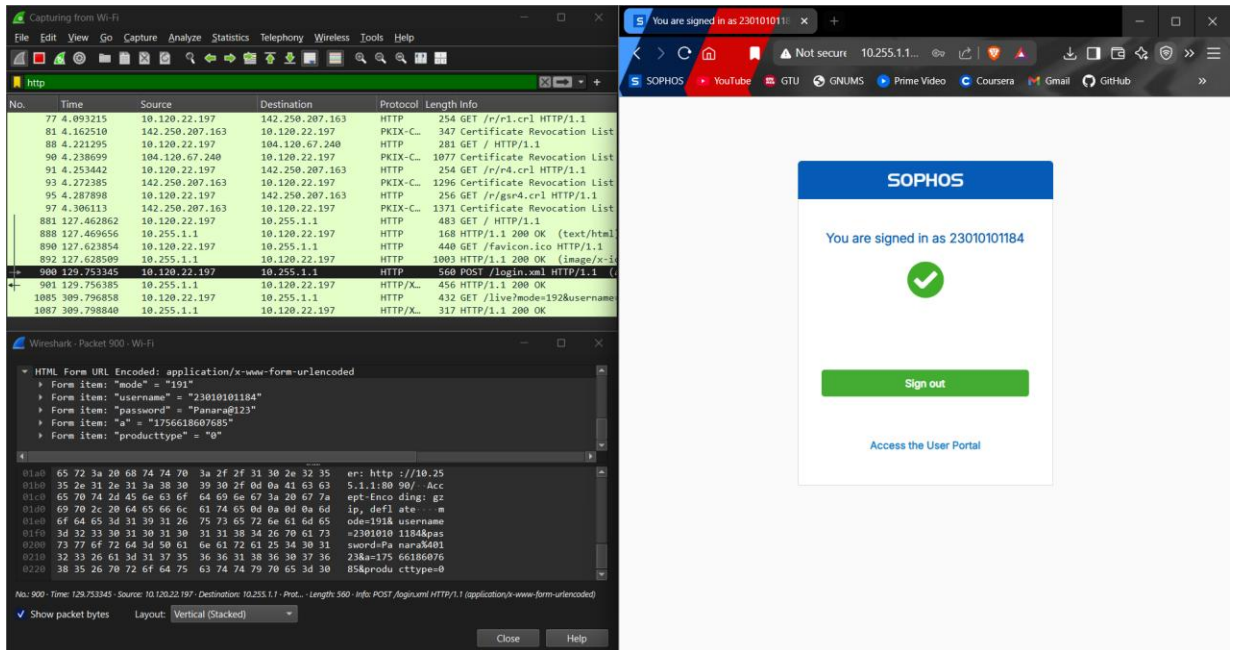
- Frame 6: 80 bytes on wire (640 bits), 80 bytes captured on interface (640 bits) on Wi-Fi
- Ethernet II, Src: Intel_08:00:00:00:00:00, Dst: Intel_08:00:00:00:00:00
- Internet Protocol Version 4, Src: 10.120.22.197, Dst: 8.8.8.8
- User Datagram Protocol, Src Port: 59000, Dst Port: 53
- Domain Name System (query)

The packet bytes pane shows the raw data in hexadecimal and ASCII. On the right is a Windows Command Prompt window showing the execution of the following commands:

```
C:\Users\WELCOME>color 70
C:\Users\WELCOME>nslookup openai.com
Server: dns.google
Address: 8.8.8.8

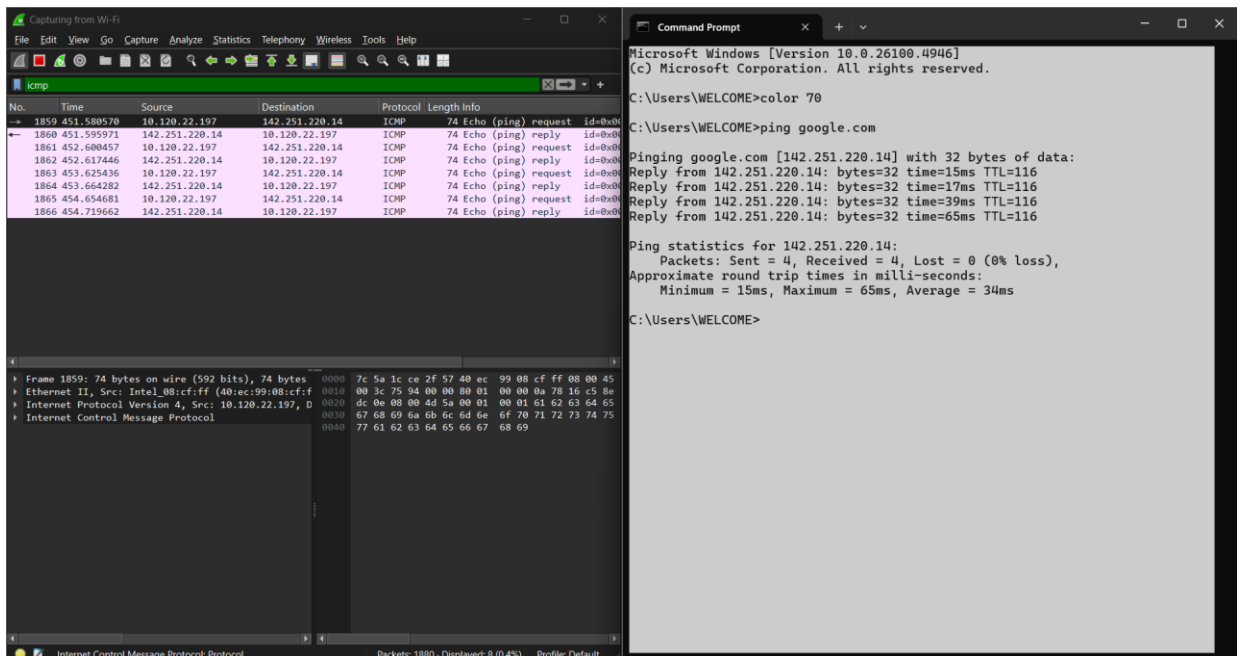
Non-authoritative answer:
Name: openai.com
Addresses: 104.18.33.45
          172.64.154.211
C:\Users\WELCOME>
```

Date: 30/07/2025



The top screenshot shows a Wireshark packet capture of an HTTP login request to the Sophos portal. The packet list shows a POST request to /login.xml. The packet details pane shows the HTML form data, including the username '23010101184' and password 'Panara@123'. The packet bytes pane shows the raw data of the POST request.

The bottom screenshot shows a web browser window displaying the Sophos login page. The page shows a green checkmark and the message 'You are signed in as 23010101184'. A 'Sign out' button is visible, and a link to 'Access the User Portal' is at the bottom.



The left screenshot shows a Wireshark packet capture of an ICMP echo request (ping) to google.com. The packet list shows a single ICMP Echo (ping) request. The packet details pane shows the ICMP header and the payload. The packet bytes pane shows the raw data of the ICMP request.

The right screenshot shows a Windows Command Prompt window. The user has entered the command 'ping google.com'. The output shows the ping statistics for google.com, including the IP address 142.251.220.14, the number of bytes sent and received, the time taken for the round trip, and the TTL value.



DARSHAN INSTITUTE OF ENGINEERING & TECHNOLOGY

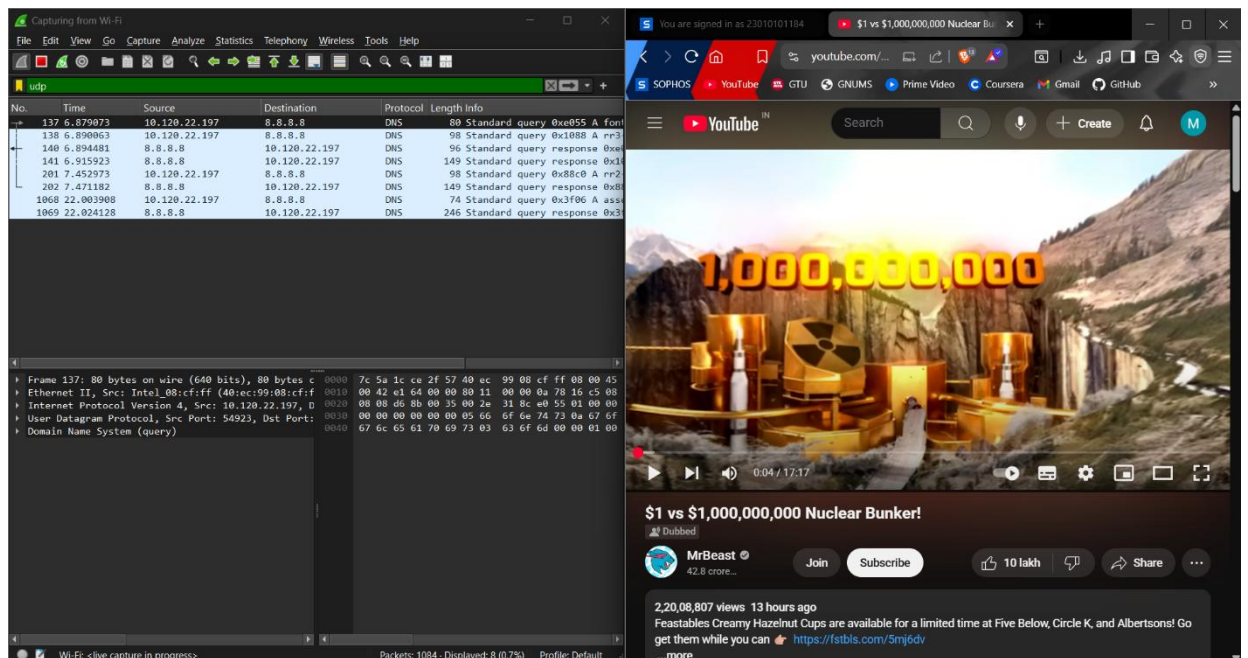
Semester 5th | Practical Assignment | Computer Networks (2301CS501)

Date: 30/07/2025

The image shows a Wireshark packet capture window on the left and a browser window on the right. The Wireshark window displays a list of captured packets, with the selected packet (No. 1856) showing details for the Transmission Control Protocol (TCP) and the Hypertext Transfer Protocol (HTTP). The browser window shows the Darshan University Learning Management System (LMS) interface, displaying the course details for Semester 5, including the course code 2301CS402 - Design and Analysis of Algorithm and 2301CS501 - Computer Network.

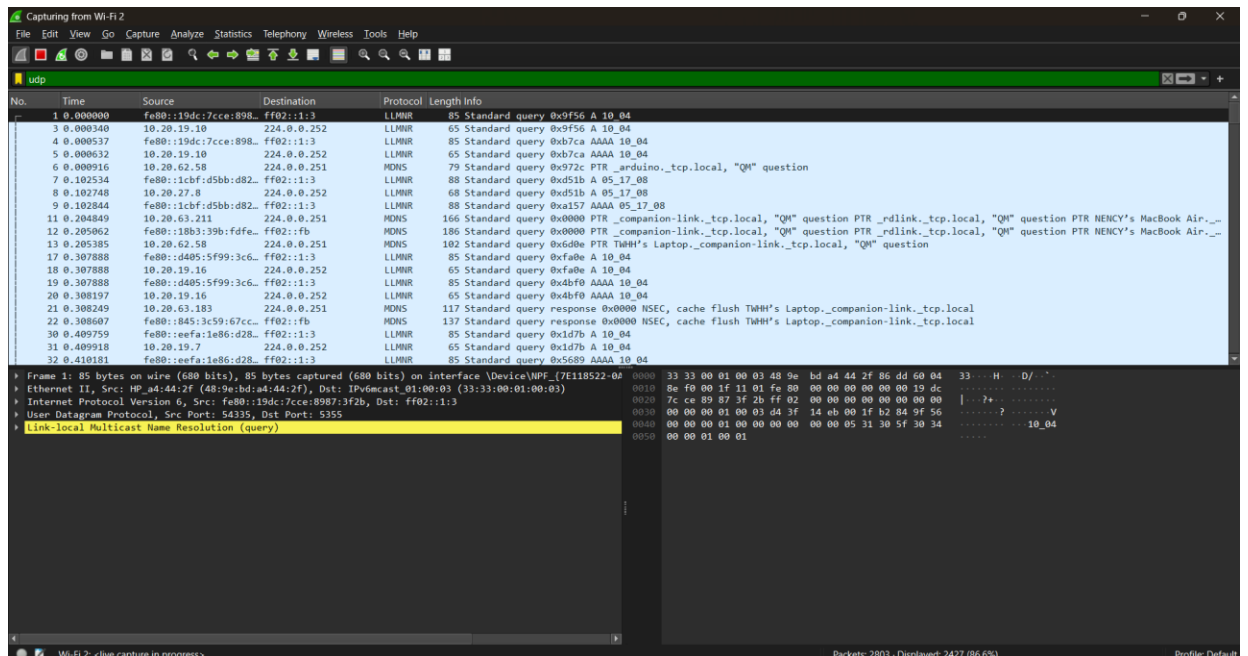
The image shows a Wireshark packet capture window on the left and a browser window on the right. The Wireshark window displays a list of captured packets, with the selected packet (No. 5378) showing details for the Transmission Control Protocol (TCP) and the Hypertext Transfer Protocol (HTTP). The browser window shows the Darshan University Learning Management System (LMS) interface, displaying the course details for Semester 5, including the course code 2301CS402 - Design and Analysis of Algorithm and 2301CS501 - Computer Network.

Date: 30/07/2025



The image shows a Wireshark packet capture of a DNS query. The packet list shows a query for '0x855 A 10_04' from source 10.120.22.197 to destination 8.8.8.8. The packet details show the query structure, including the question and the response. The packet bytes show the raw data of the query and response.

The image also shows a YouTube video player with the title '\$1 vs \$1,000,000,000 Nuclear Bunker!'. The video has 2,20,08,807 views and was uploaded 13 hours ago. The video content shows a nuclear bunker with a large '1,000,000,000' overlay.



The image shows a Wireshark packet capture of a DNS query. The packet list shows a query for '0x9F56 A 10_04' from source 10.20.19.10 to destination 224.0.0.252. The packet details show the query structure, including the question and the response. The packet bytes show the raw data of the query and response.

The image also shows a YouTube video player with the title '\$1 vs \$1,000,000,000 Nuclear Bunker!'. The video has 2,20,08,807 views and was uploaded 13 hours ago. The video content shows a nuclear bunker with a large '1,000,000,000' overlay.

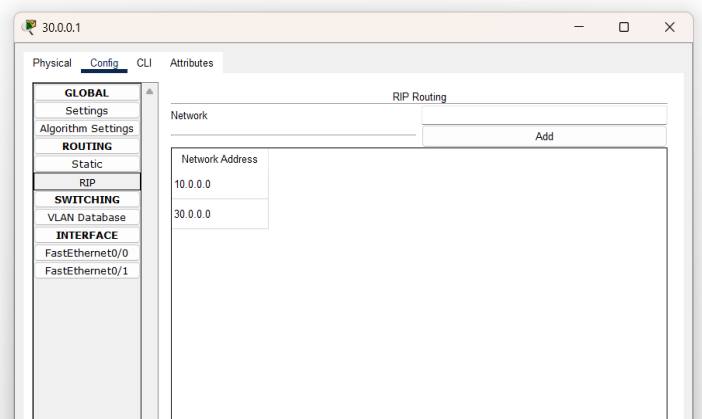
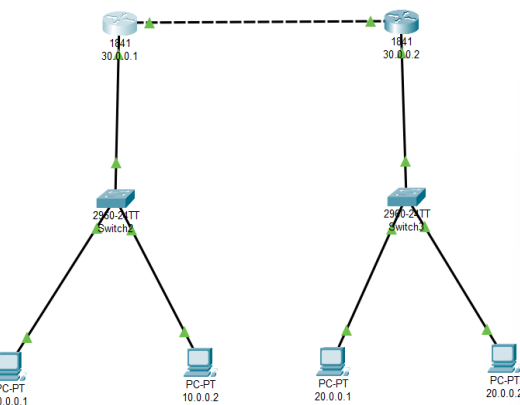
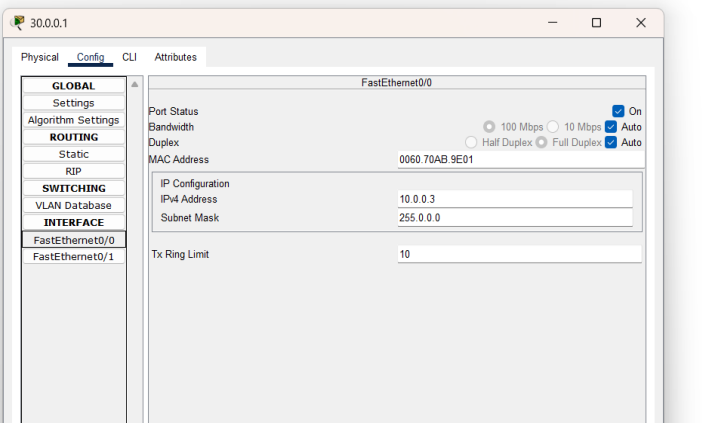
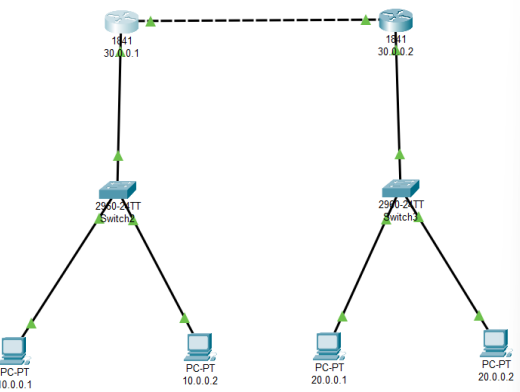
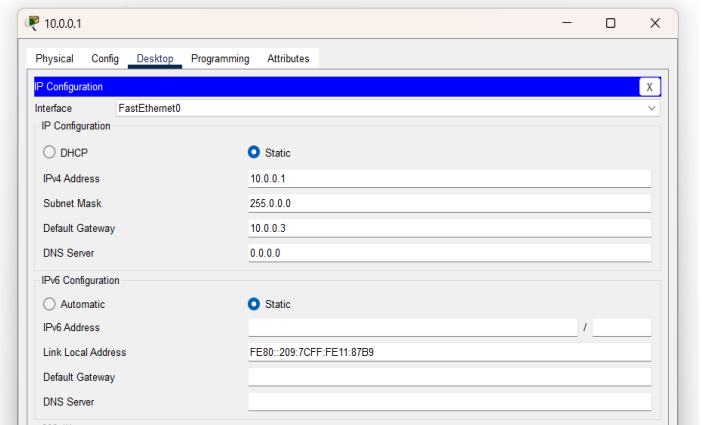
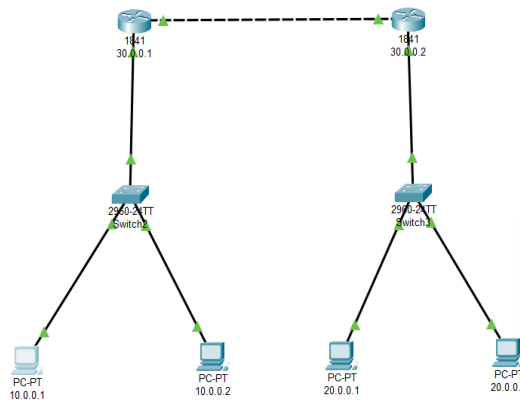
Date: 20/08/2025

Lab Practical #11:

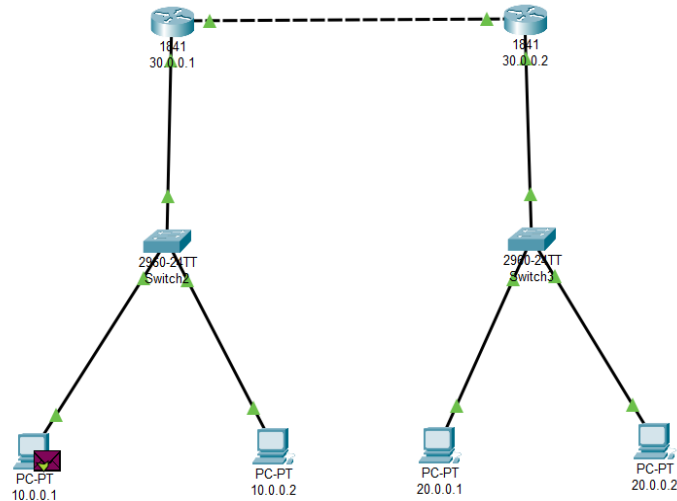
Study the concept of routing using packet tracer. (Dynamic Routing)

Practical Assignment #11:

1. Connect the two different networks based on the calculated IP addresses and subnet using a packet tracer.



Date: 20/08/2025



Simulation Panel		
Event List		
Vis.	Time(sec)	Last Device
	0.000	--
	0.001	10.0.0.1
	0.002	Switch2
	0.003	30.0.0.1
	0.004	30.0.0.2
	0.005	Switch3
	0.006	20.0.0.1
	0.007	Switch3
	0.008	30.0.0.2
	0.009	30.0.0.1
	0.010	Switch2

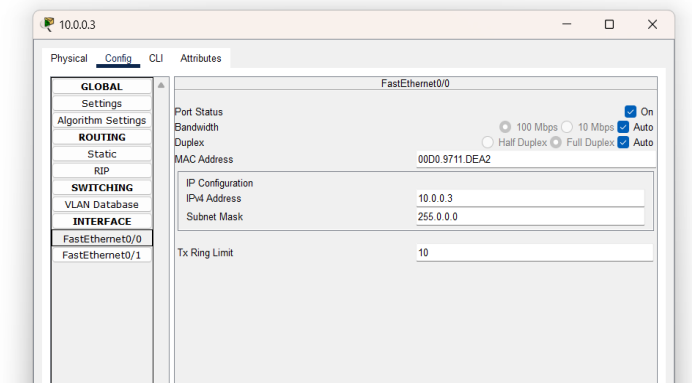
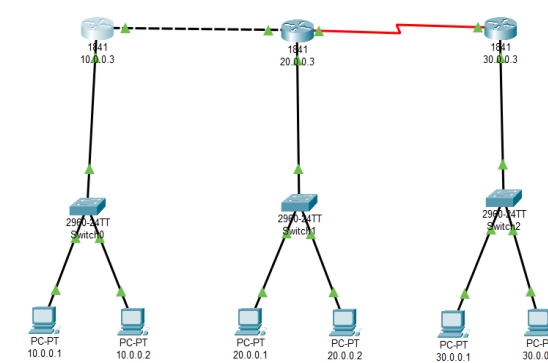
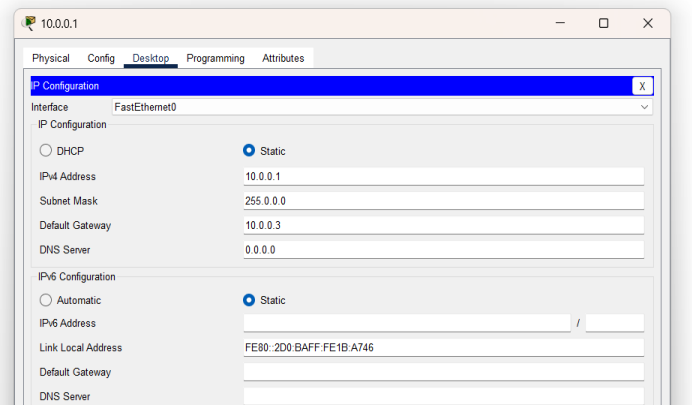
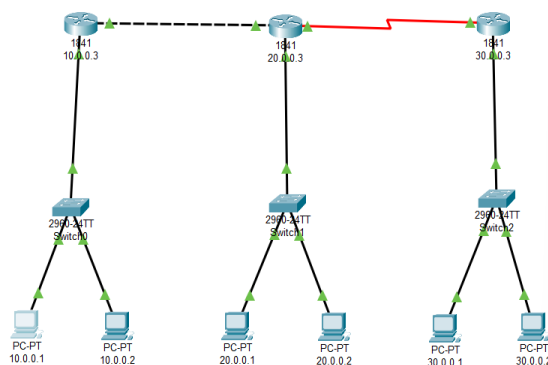
Reset Simulation ☒ Constant Delay Captured to: 0.010 s

Play Controls

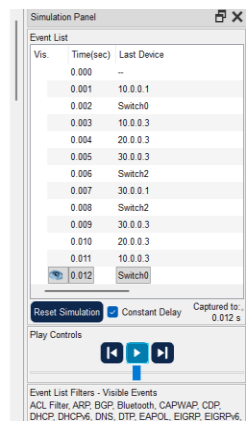
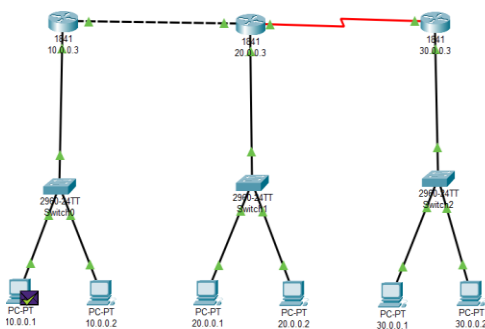
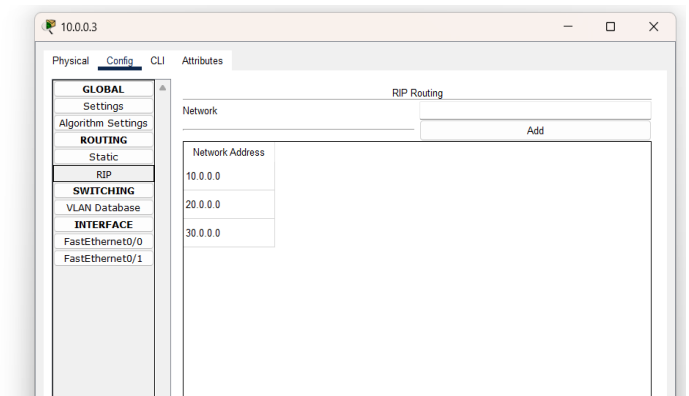
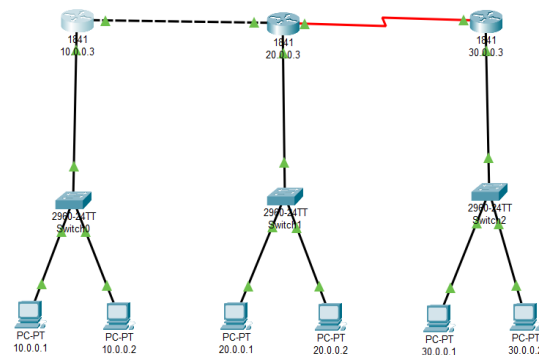
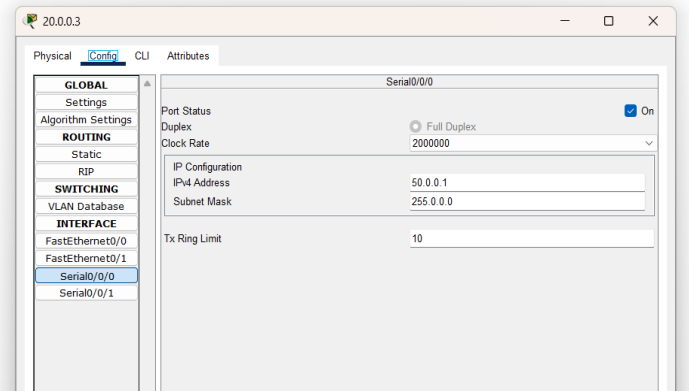
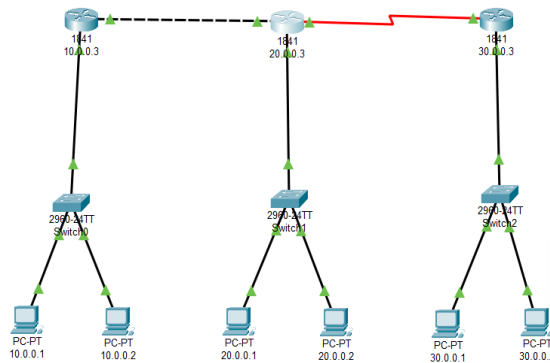
Event List Filters - Visible Events

ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, DTP, EAPOL, EIGRP, EIGRPv6, FTP, H.323, HSRP, HSRPv6, HTTP, HTTPS, ICMP, ICMPv6, IPsec, ISAKMP, IoT, IoT TCP, LACP, LLDP, MIB, NETFLOW, NTP, OSPF, OSPFv6, RADIUS, RDP

- Connect the three different networks based on the calculated IP addresses and subnet using a packet tracer.



Date: 20/08/2025



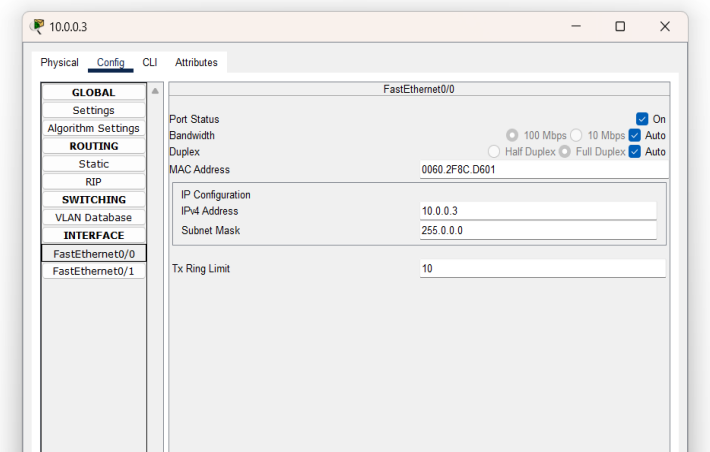
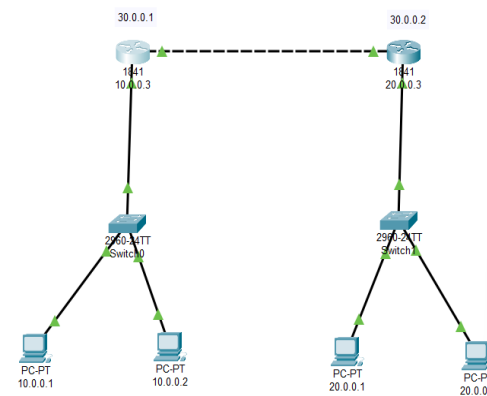
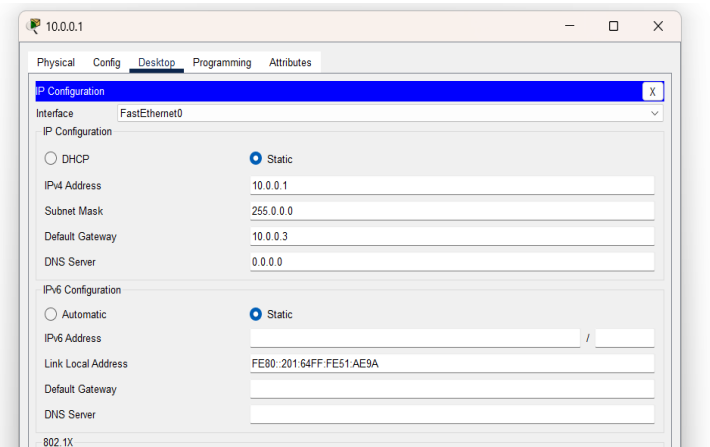
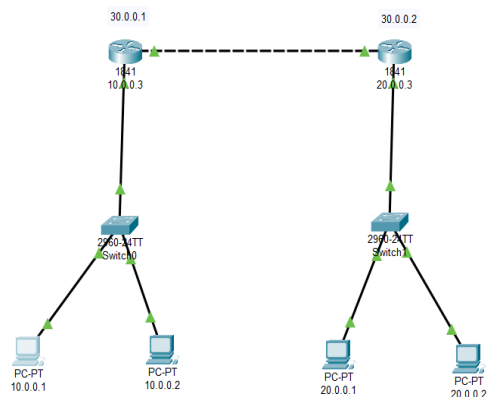
Date: 27/08/2025

Lab Practical #12:

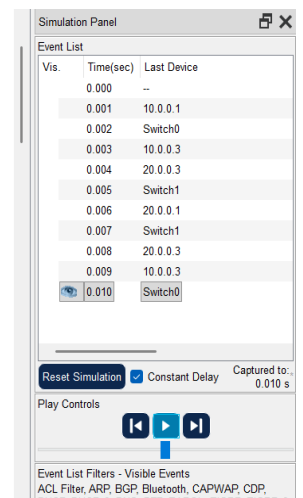
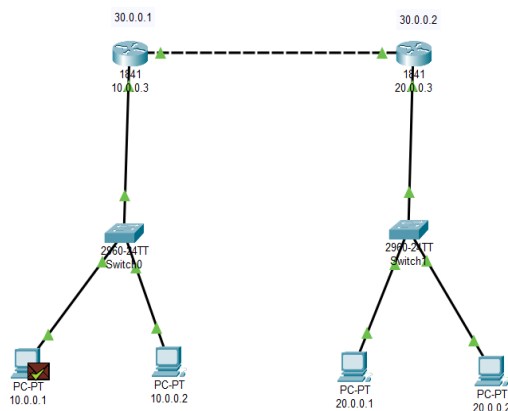
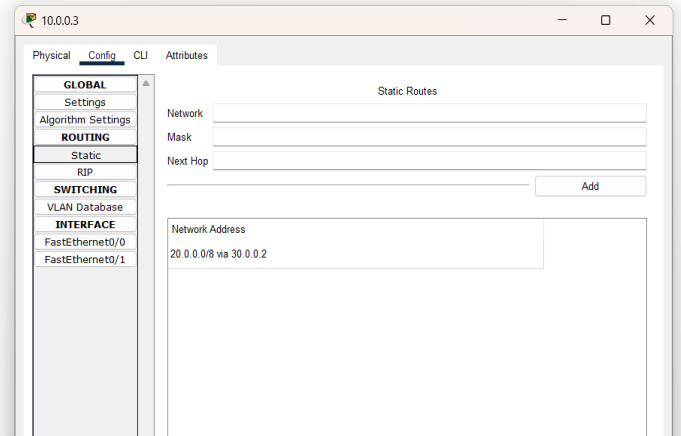
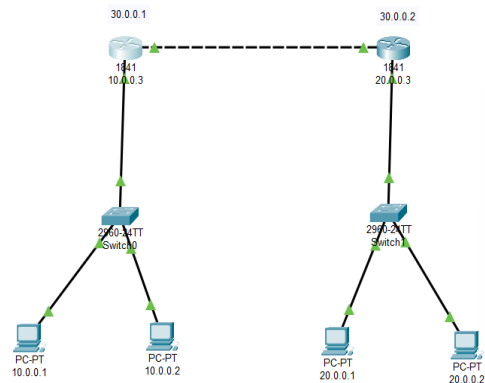
Study the concept of routing using packet tracer. (Static Routing)

Practical Assignment #12:

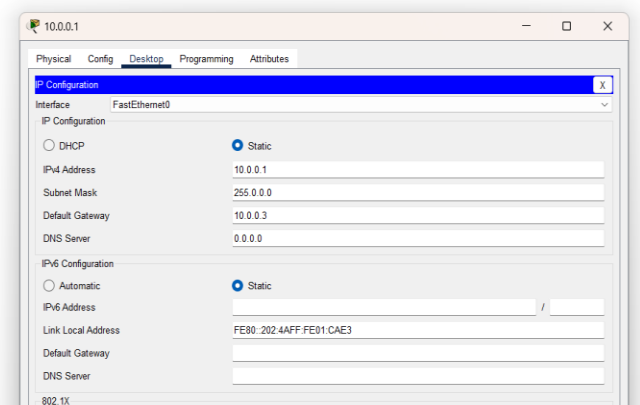
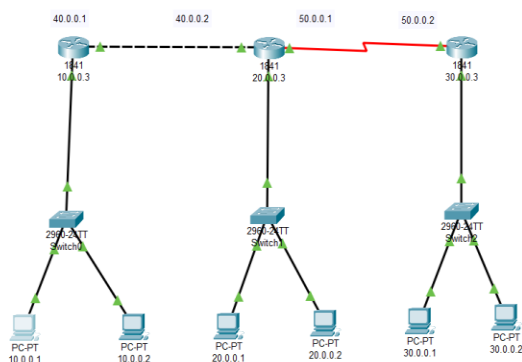
1. Connect the two different networks based on the calculated IP addresses and subnet using a packet tracer.



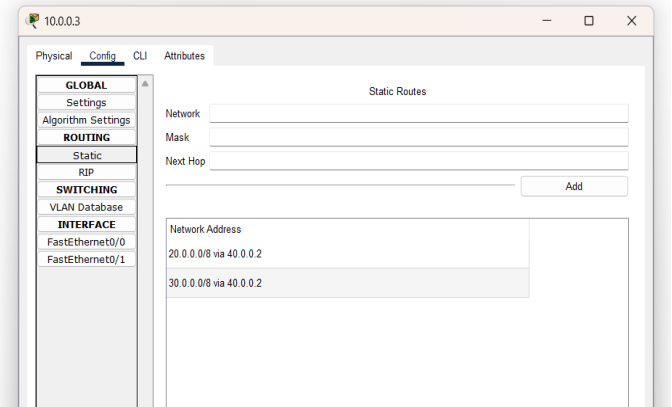
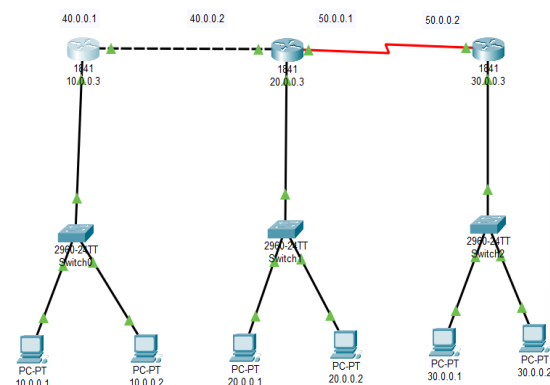
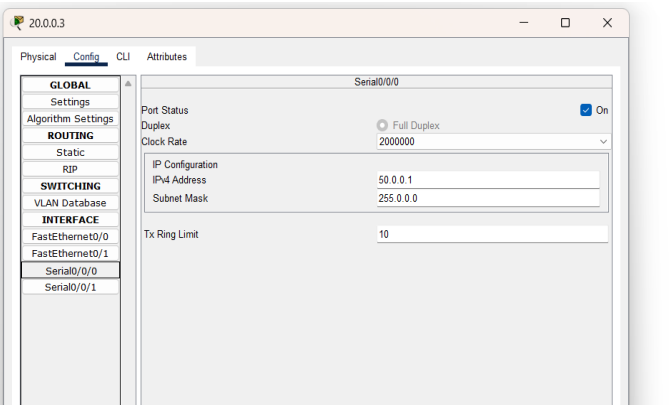
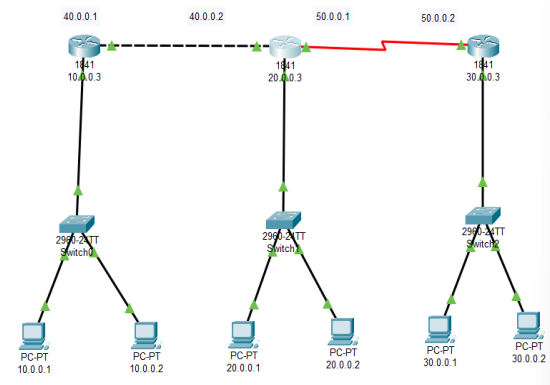
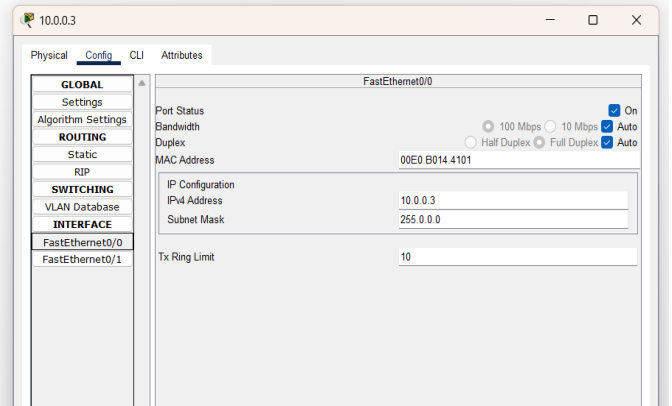
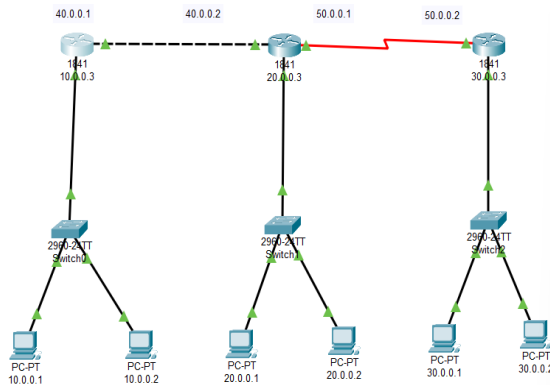
Date: 27/08/2025



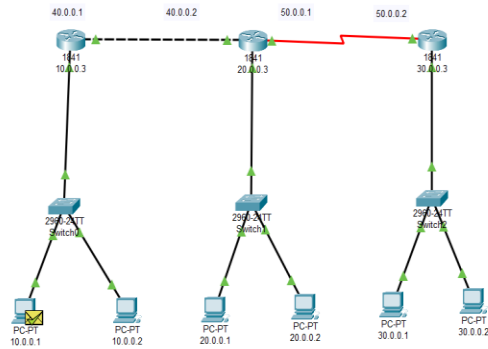
2. Connect the three different networks based on the calculated IP addresses and subnet using a packet tracer.



Date: 27/08/2025



Date: 27/08/2025



Simulation Panel		
Event List		
Vis.	Time(sec)	Last Device
	0.000	-
	0.001	10.0.0.1
	0.002	Switch0
	0.003	10.0.0.3
	0.004	20.0.0.3
	0.005	30.0.0.3
	0.006	Switch2
	0.007	30.0.0.1
	0.008	Switch2
	0.009	30.0.0.3
	0.010	20.0.0.3
	0.011	10.0.0.3
	0.012	Switch0

Reset Simulation ☒ Constant Delay Captured to: 0.012 s

Play Controls

Event List Filters - Visible Events
ACL Filter, ARP, BGP, Bluetooth, CAPWAP, CDP, DHCP, DHCPv6, DNS, OTT, EAPOL, EIGRP, EIGRPv6, ...

Date: 03/09/2025

Lab Practical #13:

To develop network using distance vector routing protocol and link state routing protocol.

Practical Assignment #13:

1. C/Java Program: Distance Vector Routing Algorithm using Bellman Ford's Algorithm.

```
import java.util.Scanner;

public class DistanceVectorRouting {
    private static final int INF = 9999;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);

        System.out.print("Enter the number of routers: ");
        int numRouters = scanner.nextInt();

        int[][] costMatrix = new int[numRouters][numRouters];
        System.out.println("Enter the cost matrix (use " + INF + " for infinity):");
        for (int i = 0; i < numRouters; i++) {
            for (int j = 0; j < numRouters; j++) {
                costMatrix[i][j] = scanner.nextInt();
            }
        }

        int[][] distanceVector = new int[numRouters][numRouters];
        int[][] nextHop = new int[numRouters][numRouters];

        for (int i = 0; i < numRouters; i++) {
            for (int j = 0; j < numRouters; j++) {
                distanceVector[i][j] = costMatrix[i][j];
                nextHop[i][j] = (costMatrix[i][j] != INF && i != j) ? j : -1;
            }
        }
    }
}
```


Date: 03/09/2025

```
}

boolean updated;
do {
    updated = false;
    for (int i = 0; i < numRouters; i++) {
        for (int j = 0; j < numRouters; j++) {
            for (int k = 0; k < numRouters; k++) {
                if (distanceVector[i][k] + distanceVector[k][j] < distanceVector[i][j]) {
                    distanceVector[i][j] = distanceVector[i][k] + distanceVector[k][j];
                    nextHop[i][j] = nextHop[i][k];
                    updated = true;
                }
            }
        }
    }
} while (updated);

System.out.println("\nFinal Distance Vector Table:");
for (int i = 0; i < numRouters; i++) {
    System.out.println("Router " + (i + 1) + ":");
    for (int j = 0; j < numRouters; j++) {
        if (distanceVector[i][j] == INF) {
            System.out.print("INF ");
        } else {
            System.out.print((distanceVector[i][j] + 1) + " ");
        }
    }
    System.out.println();
}

scanner.close();
}
```

Date: 03/09/2025

}

2. C/Java Program: Link state routing algorithm.

```
import java.util.*;
public class Dijkstra {
    static final int INF = Integer.MAX_VALUE;
    static int findKey(boolean[] visited, int[] distance, int V) {
        int min = INF;
        int key = -1;

        for (int i = 0; i < V; i++) {
            if (!visited[i] && distance[i] < min) {
                min = distance[i];
                key = i;
            }
        }
        return key;
    }
    static void dijkstra(int[][] graph, int src) {
        int V = graph.length;
        boolean[] visited = new boolean[V];
        int[] distance = new int[V];
        Arrays.fill(distance, INF);

        distance[src] = 0;

        for (int i = 0; i < V - 1; i++) {
            int u = findKey(visited, distance, V);
            if (u == -1) break;

            visited[u] = true;

            for (int v = 0; v < V; v++) {
```

Date: 03/09/2025

```
        if (graph[u][v] != 0 && !visited[v] && distance[u] != INF
            && distance[v] > distance[u] + graph[u][v]) {
            distance[v] = distance[u] + graph[u][v];
        }
    }

    System.out.println("\nShortest distances from node " + src + ":");
    for (int i = 0; i < V; i++) {
        if (distance[i] == INF)
            System.out.println("Node " + i + ": INF");
        else
            System.out.println("Node " + i + ": " + distance[i]);
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter number of vertices: ");
        int V = sc.nextInt();

        int[][] graph = new int[V][V];
        System.out.println("Enter adjacency matrix (0 if no edge):");
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                graph[i][j] = sc.nextInt();
            }
        }

        System.out.print("Enter source node (0 to " + (V - 1) + "): ");
        int src = sc.nextInt();

        System.out.println("\nGraph:");
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++)
                System.out.print(graph[i][j] + "\t");
            System.out.println();
        }
        dijkstra(graph, src);
    }
```



DARSHAN INSTITUTE OF ENGINEERING & TECHNOLOGY
Semester 5th | Practical Assignment | Computer Networks (2301CS501)

Date: 03/09/2025

}

}

Date: 10/09/2025

Lab Practical #14:

Implementation of parity bit check Using C/Java language with example.

Practical Assignment #14:

1. C/Java Program: Implementation of parity bit check Using C/Java language.

```
import java.util.Scanner;

public class ParityBitCheck {

    public static String addParityBit(String dataBits, String parityType) {
        int countOnes = 0;
        for (char bit : dataBits.toCharArray()) {
            if (bit == '1') countOnes++;
        }

        char parityBit;

        if (parityType.equalsIgnoreCase("even")) {
            parityBit = (countOnes % 2 == 0) ? '0' : '1';
        } else {
            parityBit = (countOnes % 2 == 0) ? '1' : '0';
        }

        return dataBits + parityBit;
    }

    public static boolean checkParity(String data, String parityType) {
        char parityBit = data.charAt(data.length() - 1);
        String dataBits = data.substring(0, data.length() - 1);

        int countOnes = 0;
        for (char bit : dataBits.toCharArray()) {
            if (bit == '1') countOnes++;
        }
    }
}
```

Date: 10/09/2025

```
}

boolean isValid;
if (parityType.equalsIgnoreCase("even")) {
    isValid = ((countOnes % 2 == 0) && parityBit == '0') ||
              ((countOnes % 2 == 1) && parityBit == '1');
} else {
    isValid = ((countOnes % 2 == 0) && parityBit == '1') ||
              ((countOnes % 2 == 1) && parityBit == '0');
}

return isValid;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);

    System.out.print("Enter binary data (without parity bit): ");
    String dataBits = scanner.nextLine();

    System.out.print("Enter parity type (even/odd): ");
    String parityType = scanner.nextLine();

    String dataWithParity = addParityBit(dataBits, parityType);
    System.out.println("Data with " + parityType + " parity bit: " +
dataWithParity);

    if (checkParity(dataWithParity, parityType)) {
        System.out.println("Parity check PASSED.");
    } else {
        System.out.println("Parity check FAILED.");
    }
}
```



Date: 10/09/2025

```
scanner.close();  
}  
}
```



Date: 10/09/2025

Lab Practical #15:

Implementation of parity bit check Using C/Java language with example.

Practical Assignment #15:

C/Java Program: Implementation of Bit stuffing Using C/Java language.

```
import java.util.Scanner;
```

```
public class BitStuffing {  
  
    public static String stuffBits(String data) {  
        StringBuilder stuffed = new StringBuilder();  
        int count = 0;  
  
        for (char bit : data.toCharArray()) {  
            stuffed.append(bit);  
            if (bit == '1') {  
                count++;  
                if (count == 5) {  
                    stuffed.append('0');  
                    count = 0;  
                }  
            } else {  
                count = 0;  
            }  
        }  
  
        return stuffed.toString();  
    }  
}
```




Date: 10/09/2025

```
public static void main(String[] args) {  
    Scanner scanner = new Scanner(System.in);  
  
    System.out.print("Enter binary data: ");  
    String data = scanner.nextLine();  
  
    String stuffed = stuffBits(data);  
    System.out.println("Stuffed Data: " + stuffed);  
    scanner.close();  
}  
}
```

- 1. Enter the binary data: 011111101111110**
Bit-stuffed data: 01111101011111010
- 2. Enter the binary data: 111110111111**
Bit-stuffed data: 1 1 1 1 1 0 0 1 1 1 1 0 1